

ADVANCED GUIDE

AI · ML · Deep Learning

Generative AI

Complete Reference for Engineers & Interviews

The AI Ecosystem



Topics Covered

10 Sections

Formulas

20+ Key

Code Snippets

30+ Examples

Interview Q&A

Included

2025 – 2026 · Generative AI Edition



TABLE OF CONTENTS

- [01 Artificial Intelligence — Foundations](#)
- [02 Machine Learning — Concepts, Algorithms & Formulas](#)
- [03 Deep Learning & Neural Networks](#)
- [04 Generative AI — Complete Deep Dive](#)
- [05 Large Language Models \(LLMs\)](#)
- [06 Prompt Engineering — Advanced Techniques](#)
- [07 Retrieval Augmented Generation \(RAG\)](#)
- [08 AI Agents & Advanced GenAI Concepts](#)
- [09 GenAI Tools, Frameworks & Ecosystem](#)
- [10 Key Formulas Cheat Sheet](#)

What is Artificial Intelligence (AI)?

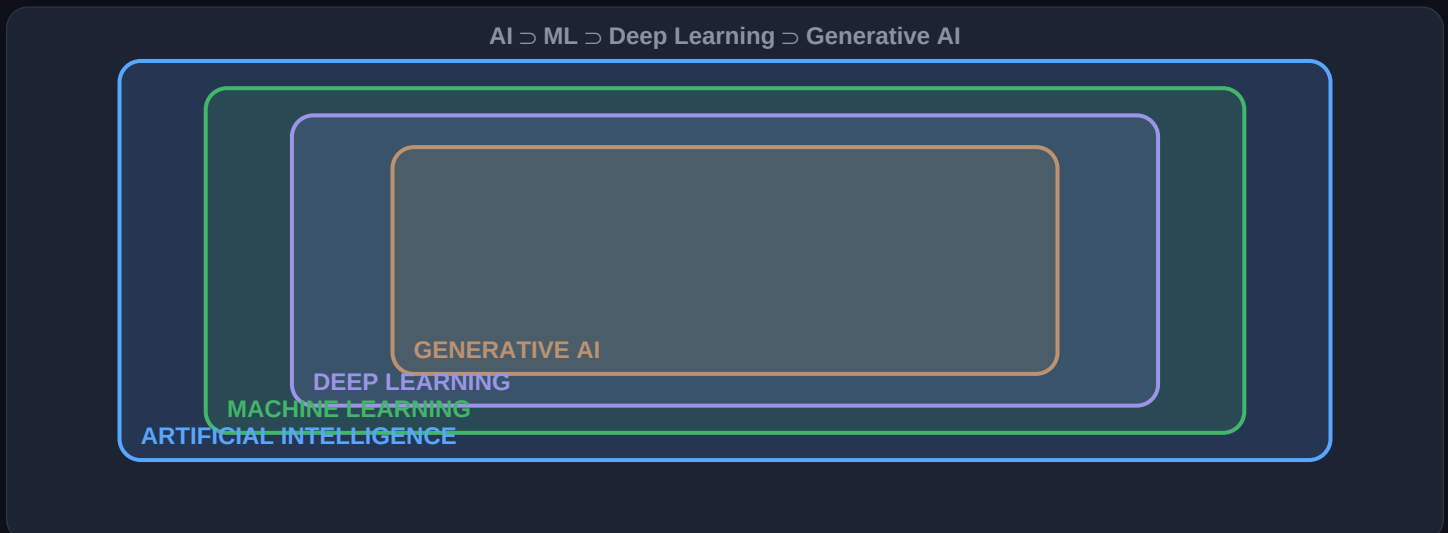
Artificial Intelligence is the science and engineering of creating computer systems that can perform tasks normally requiring human intelligence — such as learning, reasoning, problem-solving, language understanding, and perception.

“

AI is the new electricity. Just as electricity transformed almost everything 100 years ago, today I actually have a hard time thinking of an industry that AI will not transform.

— Andrew Ng

The AI Hierarchy



Types of AI — Definitions

Narrow AI (ANI)

AI designed to do ONE specific task really well — like face recognition or spam filtering. All AI we use today is Narrow AI.

General AI (AGI)

AI that can do anything a human can — reasoning, learning, creativity across all domains. It does not exist yet; still being researched.

Super AI (ASI)

Hypothetical AI that surpasses all human intelligence combined. Pure science fiction for now — does not exist.

Reactive AI

Has no memory — only reacts to what is in front of it right now. Example: the chess engine that beat Kasparov.

Limited Memory AI

Uses recent past data to make better decisions. Most current AI — like ChatGPT and self-driving cars — works this way.

Theory of Mind AI

AI that understands human emotions and social context. Still in the research phase — does not fully exist yet.

Self-Aware AI

AI with consciousness, feelings, and self-awareness. Completely hypothetical — pure science fiction for now.

Interview Questions & Answers

Q: What is AI in one line?

AI is giving computers the ability to learn from data and make smart decisions — without being manually programmed for every rule.

Q: What is the Turing Test?

A test where a human chats with both a person and an AI — if they cannot tell the difference, the AI passes. Proposed by Alan Turing in 1950.

Q: Is ChatGPT Narrow AI or General AI?

Narrow AI. It is excellent at language tasks but cannot drive a car or recognize faces — it is specialized for one domain.

Library: Using AI APIs in Python

openai · Python API Example

```
# OpenAI API — Quick Start
import openai

client = openai.OpenAI(api_key='your-key')
response = client.chat.completions.create(
    model='gpt-4o',
    messages=[{'role': 'user', 'content': 'What is AI?'}]
)
print(response.choices[0].message.content)
```

anthropic · Claude API Example

```
# Anthropic Claude API
import anthropic

client = anthropic.Anthropic(api_key='your-key')
message = client.messages.create(
    model='claude-sonnet-4-20250514',
    max_tokens=1024,
    messages=[{'role': 'user', 'content': 'Explain AI'}]
)
print(message.content[0].text)
```

What is Machine Learning (ML)?

Machine Learning is a subset of AI that gives computers the ability to learn from data without being explicitly programmed. Instead of writing rules manually, you feed data and correct answers to an algorithm — and it learns the rules automatically.

“

Machine learning is the field of study that gives computers the ability to learn without being explicitly programmed.

— Arthur Samuel, 1959

The 3 Types of Machine Learning



Supervised Learning

The model learns from labeled data — you give it inputs with correct answers and it learns the pattern. Example: spam detection, house price prediction.

Unsupervised Learning

The model finds hidden patterns in data with no labels provided. Example: grouping customers into segments without being told the categories.

Reinforcement Learning

The model learns by trial and error — gets rewards for good actions, penalties for bad ones. Used in game-playing AI like AlphaGo.

Semi-Supervised Learning

Uses a small amount of labeled data combined with a large amount of unlabeled data. Useful when labeling is expensive.

Self-Supervised Learning

The model generates its own labels from raw data — like predicting the next word in a sentence. This is exactly how GPT models are pre-trained.

Core Algorithms — Definitions

Linear Regression

Predicts a continuous number by fitting the best straight line through data. Used for house prices, salary estimation, and temperature forecasting.

Logistic Regression

Predicts a yes/no outcome as a probability between 0 and 1. Used for spam detection, disease diagnosis, and binary classification tasks.

Decision Tree

Makes decisions through a flowchart of yes/no questions. Easy to understand and explain — you can see exactly why it made a decision.

Random Forest

Builds hundreds of Decision Trees and takes a majority vote. More accurate and reliable than a single tree — used in fraud detection and credit scoring.

Support Vector Machine (SVM)

Finds the best boundary line that separates two classes with maximum margin. Works well for text and image classification.

K-Nearest Neighbors (KNN)

Classifies a data point by looking at its K closest neighbors and taking a majority vote. Simple, intuitive, no training required.

K-Means Clustering

Groups data into K clusters where similar items are placed together. Used for customer segmentation and pattern discovery.

Naive Bayes

A fast probabilistic classifier based on Bayes Theorem. Very popular for spam filtering and sentiment analysis.

Gradient Boosting / XGBoost

Builds trees sequentially where each tree fixes the mistakes of the previous one. Highest accuracy on tabular data — wins most Kaggle competitions.

PCA (Principal Component Analysis)

Reduces the number of features in your data by keeping only the most important information. Used for visualization and speeding up models.

Key Formulas

LINEAR REGRESSION

$$y = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$$

y=predicted w=weights x=features b=bias

SIGMOID / LOGISTIC

$$\text{sigma}(z) = 1 / (1 + e^{(-z)}) \quad \text{where } z = w \cdot x + b$$

Output: 0 to 1 | Threshold 0.5: >0.5 → Class 1, <0.5 → Class 0

MEAN SQUARED ERROR

$$\text{MSE} = (1/n) * \text{Sum}(y_i - \hat{y}_i)^2$$

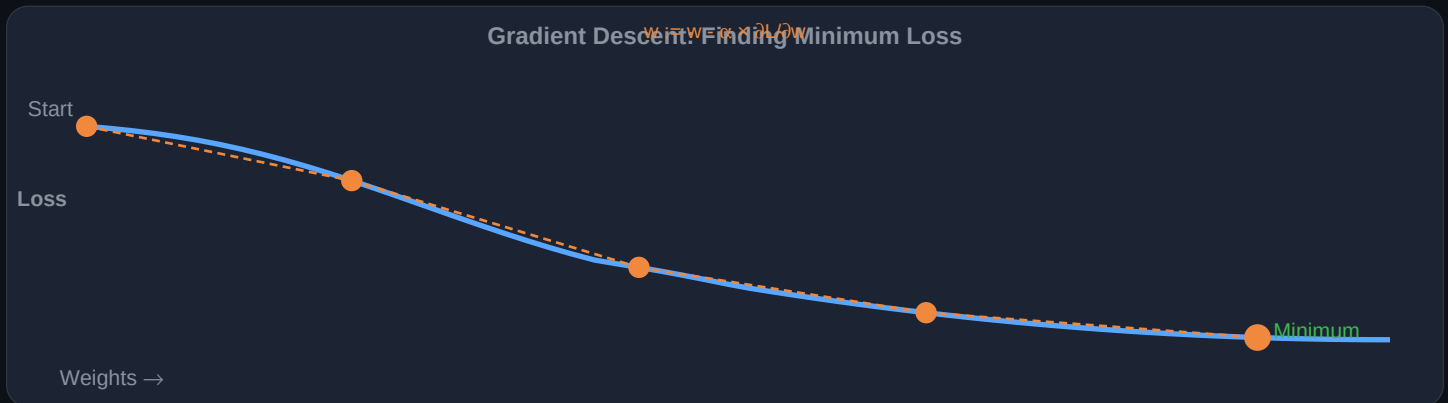
y_i =actual $\hat{y}_i$ =predicted n =samples | Lower is better

F1 SCORE

$$\text{F1} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

Best metric for imbalanced datasets | Range: 0 (worst) to 1 (best)

Gradient Descent — Visual



GRADIENT DESCENT UPDATE

$$w := w - \alpha * (dL/dw)$$

α =learning rate dL/dw =gradient | Iteratively minimize loss

Overfitting & Underfitting — Definitions

Underfitting (High Bias)

The model is too simple to capture the pattern — performs poorly even on training data. Fix: use a more complex model or add more features.

Overfitting (High Variance)

The model memorizes training data but fails on new data. Like mugging answers without understanding. Fix: regularization, more data, early stopping.

Good Fit

The model generalizes well — performs strongly on both training data and new unseen data. This is the goal of every ML project.

L1 Regularization (Lasso)

Prevents overfitting by penalizing large weights — can zero out unimportant features entirely, acting as automatic feature selection.

L2 Regularization (Ridge)

Prevents overfitting by shrinking all weights smaller — no feature dominates. Gentler than L1 and very commonly used.

Cross-Validation (k-fold)

Splits data into K parts and trains/tests K times to get a reliable estimate of model performance — more honest than a single train-test split.

Python Code — Machine Learning with Scikit-learn

scikit-learn · Random Forest Classifier

```
# pip install scikit-learn pandas numpy
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
import pandas as pd

# Load data
df = pd.read_csv('data.csv')
X = df.drop('target', axis=1)
y = df['target']

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Train Random Forest
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Evaluate
y_pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```


What is Deep Learning?

Deep Learning is a subset of ML that uses Artificial Neural Networks with multiple hidden layers to learn hierarchical representations from data. 'Deep' refers to the many layers — each layer learns increasingly abstract features.

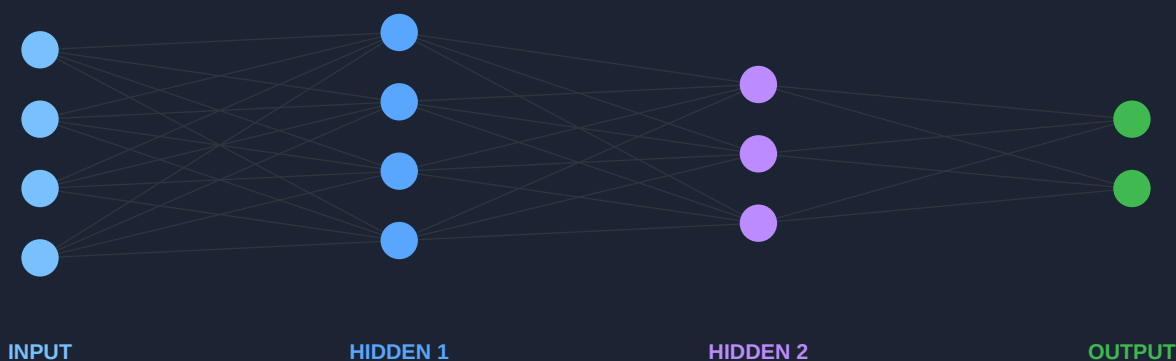
“

Deep learning is going to be able to do everything.

— Geoffrey Hinton, Godfather of Deep Learning

Neural Network Architecture

Feedforward Neural Network — Layers & Connections



Input Layer

The entry point of a neural network — receives raw data as numbers. One node per feature, pixel, or word dimension.

Hidden Layer

The middle layers where all the learning happens. More hidden layers means deeper, more powerful models — hence the name Deep Learning.

Output Layer

Produces the final prediction — a class probability for classification or a number for regression. This is where the answer comes out.

Weight (w)

A learnable number that controls the strength of a connection between neurons. Training is the process of finding the best weights.

Bias (b)

An extra learnable parameter that shifts a neuron's output — allows the model to fit data that does not pass through zero.

Activation Functions — All Definitions

Sigmoid

Squashes any number into a 0–1 range — used at the output layer for binary classification problems like yes/no decisions.

Tanh

Like Sigmoid but outputs -1 to 1. Zero-centered — trains faster than Sigmoid. Commonly used in RNNs for sequence tasks.

ReLU (Rectified Linear Unit)

If input is negative: output 0. If positive: pass through as-is. Simple, fast, and the default choice for hidden layers in most neural networks.

Leaky ReLU

A smarter version of ReLU that allows a small gradient for negative values — prevents neurons from permanently dying during training.

Softmax

Converts raw scores into probabilities that sum to 1. Used at the output layer when classifying into 3 or more categories.

GELU (Gaussian Error Linear Unit)

A smooth, modern alternative to ReLU used in Transformers like GPT and Claude. Slightly outperforms ReLU on language tasks.

Backpropagation — How Neural Nets Learn

Forward Pass

Data flows through all layers from input to output to produce a prediction. The loss is then calculated by comparing the prediction to the true answer.

Backward Pass (Backpropagation)

The error is sent backwards through the network, adjusting each weight based on how much it contributed to the mistake. This is how neural nets learn.

Chain Rule

The calculus rule that makes backpropagation possible — traces how much each weight contributed to the final error across multiple layers.

Vanishing Gradient

In deep networks, gradients shrink layer by layer until the first layers learn almost nothing. Fixed by using ReLU and residual connections.

Deep Learning Model Zoo

CNN (Convolutional Neural Network)

Uses sliding filters to detect visual patterns in images. Early layers find edges, deeper layers find objects. Powers face recognition and self-driving vision.

RNN (Recurrent Neural Network)

Processes sequences one step at a time while keeping a hidden memory of previous steps. Good for short sequences — mostly replaced by Transformers today.

LSTM (Long Short-Term Memory)

A smarter RNN with forget/input/output gates that control what to remember or discard — handles long-range dependencies much better.

GRU (Gated Recurrent Unit)

A simpler, faster version of LSTM with only 2 gates. Performs similarly to LSTM but trains quicker — good when compute is limited.

Transformer

The architecture behind all modern LLMs — processes all tokens in parallel using self-attention. Much faster and more powerful than RNNs.

GAN (Generative Adversarial Network)

Two networks compete — a Generator creates fake data, a Discriminator tries to detect it. They push each other until the Generator produces realistic output.

Diffusion Model

Learns to reverse a noise-adding process — starts from random noise and gradually cleans it into a realistic image. Powers DALL-E 3 and Stable Diffusion.

Python Code — Neural Network with PyTorch

```
# pip install torch torchvision
import torch
import torch.nn as nn
import torch.optim as optim

# Define a simple feedforward network
class NeuralNet(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(NeuralNet, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_size, num_classes)
    def forward(self, x):
        out = self.fc1(x)
        out = self.relu(out)
        out = self.fc2(out)
        return out

model = NeuralNet(input_size=784, hidden_size=256, num_classes=10)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Training loop
for epoch in range(10):
    outputs = model(X_train) # forward pass
    loss = criterion(outputs, y_train)
    optimizer.zero_grad()    # clear gradients
    loss.backward()          # backpropagation
    optimizer.step()         # update weights
    print(f'Epoch {epoch}, Loss: {loss.item():.4f}')
```

What is Generative AI?

Generative AI (GenAI) is a class of AI models that can CREATE new content — text, images, audio, video, code, and 3D objects — by learning the underlying patterns and statistical distributions of their training data.

“

We are moving from AI as a tool to AI as a creative collaborator. Generative AI doesn't just answer questions — it imagines possibilities.

— Sam Altman, OpenAI CEO

Discriminative vs Generative — Definitions

Discriminative Model

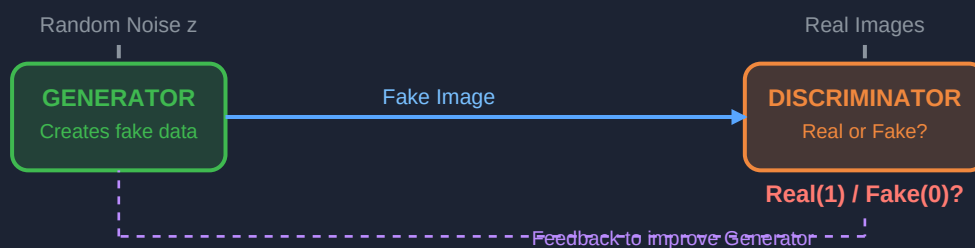
Classifies existing data into categories — learns what separates one class from another. Examples: spam filter, BERT, image classifier.

Generative Model

Learns the structure of data and can create new content from scratch — like GPT generating text or DALL-E generating images.

GAN — Generative Adversarial Network

GAN Training Loop — Generator vs Discriminator



Generator (G)

The creative half of a GAN — takes random noise and transforms it into realistic-looking data like images or audio.

Discriminator (D)

The detective half of a GAN — classifies input as real or fake, pushing the Generator to keep improving.

Minimax Objective

The GAN's training goal — Generator minimizes detectability, Discriminator maximizes detection accuracy. They compete until equilibrium.

Mode Collapse

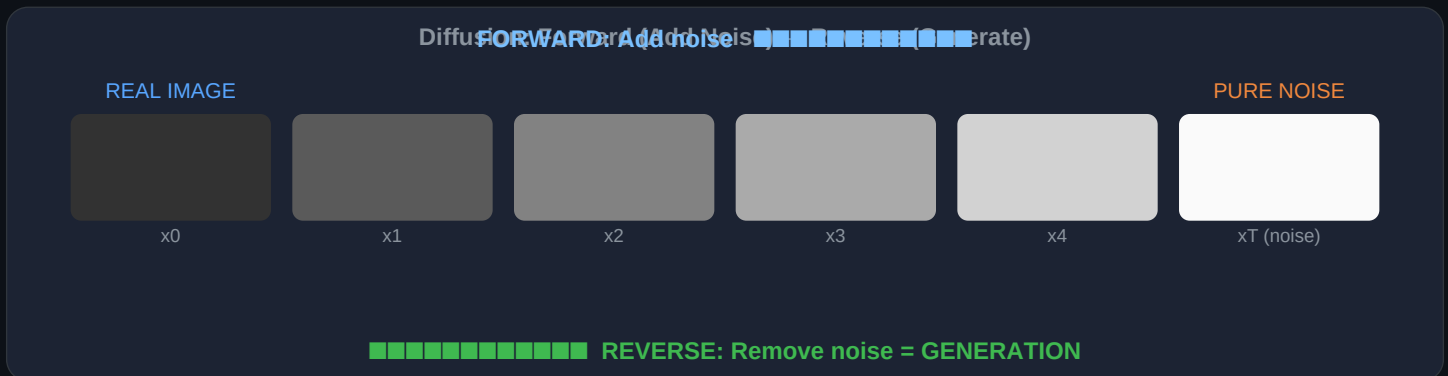
A GAN training failure where the Generator only produces one type of output instead of diverse results — a well-known training challenge.

GAN OBJECTIVE

$$\min_G \max_D [E[\log D(x)] + E[\log(1-D(G(z)))]]$$

D=Discriminator G=Generator x=real data z=random noise

Diffusion Models — Current SOTA



Forward Process

Gradually adds noise to real data over many steps until it becomes pure random noise — the model learns this process to reverse it later.

Reverse Process

Starting from pure noise, the model denoises step by step to generate a realistic image — guided by your text prompt.

Stable Diffusion

Open-source image generation model that runs locally on your own machine. Generate any image from a text prompt — free and private.

DALL-E 3 (OpenAI)

OpenAI's text-to-image model integrated into ChatGPT. Excellent at following complex descriptions and generating accurate, detailed scenes.

Midjourney

A popular paid image generator known for its stunning artistic quality. Runs through Discord — widely used by designers and creative professionals.

Types of Generative AI by Output

Text Generation

AI that produces human-like text one token at a time. Powers chatbots, writing assistants, and code generators. Examples: ChatGPT, Claude, Gemini.

Image Generation

Creates images from text prompts using diffusion or GAN models. Examples: DALL-E 3, Midjourney, Stable Diffusion.

Audio / Music Generation

AI that generates music or realistic speech from text. Suno creates full songs; ElevenLabs clones voices. Used in podcasts and content creation.

Video Generation

Creates short video clips from text prompts. OpenAI's Sora is the leading example — still improving but already impressive for ads and prototyping.

Code Generation

AI that writes working code from plain English descriptions. GitHub Copilot and Claude Code are leading examples — huge productivity boost for developers.

Multimodal Generation

A single model that handles text, images, audio, and video together. Examples: GPT-4o and Claude 3.5 — you can show them images and ask questions.

Python Code — Image Generation with Diffusers

diffusers · Stable Diffusion Image Generation

```
# pip install diffusers transformers accelerate torch
from diffusers import StableDiffusionPipeline
import torch

# Load Stable Diffusion v1.5 model
model_id = 'runwayml/stable-diffusion-v1-5'
pipe = StableDiffusionPipeline.from_pretrained(
    model_id, torch_dtype=torch.float16
)
pipe = pipe.to('cuda') # or 'mps' for Apple Silicon

# Generate image from text prompt
prompt = 'A futuristic city at sunset, cyberpunk style'
negative_prompt = 'blurry, low quality, dark'

image = pipe(
    prompt=prompt,
    negative_prompt=negative_prompt,
    num_inference_steps=50,
    guidance_scale=7.5
).images[0]

image.save('generated_city.png')
print('Image saved!')
```

What is an LLM?

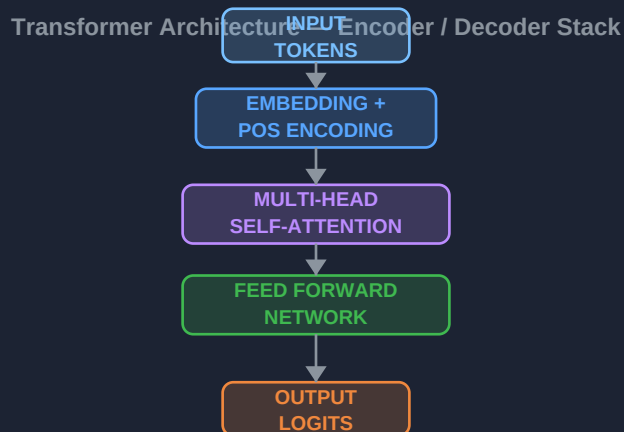
A Large Language Model (LLM) is a neural network trained on billions of words to understand and generate human language. Built on the Transformer architecture with billions to trillions of parameters, LLMs learn by predicting the next token given all previous tokens.

“

The most surprising thing about large language models is not that they work — it's that they work this well with such a simple objective.

— Ilya Sutskever, Co-founder OpenAI

Transformer Architecture



Self-Attention

Allows every token in a sequence to look at every other token simultaneously — capturing relationships like 'which word does 'it' refer to?'

Multi-Head Attention

Runs self-attention multiple times in parallel — each head captures different relationships like grammar, subject, or topic. Results are combined.

Positional Encoding

Since Transformers process all tokens at once, positional encoding injects word order information so the model knows sequence position.

Encoder

Reads and understands the full input sequence bidirectionally. Used in models like BERT for classification and understanding tasks.

Decoder

Generates text one token at a time, each new token based on all previous ones. GPT and Claude are decoder-only models.

SCALED DOT-PRODUCT ATTENTION

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) * V$$

Q=Query K=Key V=Value d_k =dimension of keys | $\sqrt{d_k}$ prevents saturation

Key LLM Concepts — All Definitions

Token

The basic unit an LLM processes — roughly 0.75 words. 'unhappy' may be 2 tokens. Costs and context limits are all measured in tokens.

Context Window

The maximum text an LLM can process at once — like working memory. Claude supports 200K tokens; Gemini 1.5 Pro supports up to 1M tokens.

Temperature

Controls randomness in generation. Low (0–0.2) = consistent, factual. High (0.7–1.0) = creative and varied. Used to balance accuracy vs. creativity.

Top-P (Nucleus Sampling)

Only samples from the top words whose combined probability reaches P — keeps output sensible while still allowing some variety.

Embeddings

Numerical vector representations of text that encode meaning — similar words have similar vectors. Foundation of semantic search and RAG.

Hallucination

When an LLM confidently states something that is factually wrong. Not intentional — it predicts likely-sounding text, not verified facts. Mitigated with RAG.

Fine-tuning

Continuing training a pre-trained model on domain-specific data to specialize it. Permanent behavior change — more powerful than prompting alone.

RLHF (Reinforcement Learning from Human Feedback)

Trains LLMs using human preferences — humans rate responses, a reward model is trained, then the LLM is optimized to score higher. Makes AI helpful and safe.

Quantization

Reduces model weight precision from 32-bit to 8 or 4-bit — drastically cuts memory usage, making large models runnable on consumer hardware.

LoRA (Low-Rank Adaptation)

Efficient fine-tuning by adding small trainable adapter layers — keeps the base model frozen. 10-100x cheaper than full fine-tuning.

Major LLMs — Comparison

GPT-4o (OpenAI)

OpenAI's flagship multimodal model — handles text, images, audio, and code. The industry benchmark. Powers ChatGPT Plus.

Claude 3.5 Sonnet (Anthropic)

Anthropic's flagship model — known for safety, long-context (200K tokens), careful reasoning, and strong coding. Powers Claude.ai.

Gemini 1.5 Pro (Google)

Google's flagship multimodal model with a 1M token context window — best for processing entire codebases or books in one request.

LLaMA 3.3 70B (Meta)

Meta's best open-source LLM — run it locally for free with full privacy. 70B parameters, competitive with commercial models.

Mistral 7B / Mixtral (Mistral AI)

Small but powerful open-source models from a French startup. Uses Mixture of Experts for efficiency — fast and great for limited hardware.

DeepSeek-V3

A GPT-4-level open-weights model from a Chinese lab — impressive on coding and math at a fraction of the cost. Shook the AI industry.

Python Code — LLM with Hugging Face Transformers

transformers · LLaMA Local Inference

```
# pip install transformers torch accelerate
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

# Load model and tokenizer
model_name = 'meta-llama/Llama-3.2-3B-Instruct'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name, torch_dtype=torch.float16
)

# Tokenize input
messages = [{'role': 'user', 'content': 'What is deep learning?'}]
inputs = tokenizer.apply_chat_template(
    messages, return_tensors='pt'
)

# Generate response
outputs = model.generate(inputs, max_new_tokens=200, temperature=0.7)
response = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(response)
```

What is Prompt Engineering?

Prompt Engineering is the discipline of designing and optimizing input instructions (prompts) to extract the most accurate, relevant, and useful outputs from AI language models. It is the highest-leverage skill for any GenAI developer — the same model with a great prompt gives 9/10 answers; with a bad prompt gives 2/10.

“

Prompting is programming in natural language. The quality of your prompt directly determines the quality of your output.

— Andrej Karpathy

All Prompting Techniques — Definitions

Zero-Shot Prompting

Ask the model to complete a task with no examples — relies entirely on its pre-trained knowledge. Works well for common, straightforward tasks.

One-Shot Prompting

Provide a single example before your real question to show the model the expected format or style.

Few-Shot Prompting

Provide 2-5 labeled examples before your question — the model learns the pattern and applies it. Significantly improves consistency and format.

Chain of Thought (CoT)

Instruct the model to reason step by step before answering. Dramatically improves accuracy on math, logic, and multi-step problems.

Zero-Shot CoT

Simply add 'Let's think step by step' to any prompt — no examples needed. This alone can double accuracy on reasoning tasks.

Role / Persona Prompting

Assign the model an expert identity before the task — 'You are a senior Python engineer...'. Improves depth, tone, and quality of the response.

System Prompt

A hidden instruction set before the conversation that defines the model's behavior, tone, and rules for the entire session.

ReAct (Reason + Act)

A prompting pattern where the model alternates between thinking and acting with tools — the foundation of how AI agents work.

Prompt Chaining

The output of one prompt becomes the input of the next — breaks complex tasks into reliable sequential steps.

Tree of Thought (ToT)

Explores multiple reasoning branches simultaneously and can backtrack — more powerful than CoT for complex planning and puzzle tasks.

Self-Consistency

Generate multiple reasoning paths for the same question and take the majority answer — improves accuracy on hard reasoning and math problems.

Constrained Output Prompting

Specify the exact output format required — like JSON only, no extra text. Essential for building AI-powered apps that process responses programmatically.

Perfect Prompt Anatomy

1. ROLE

Set the AI's expert persona. 'You are a senior Python developer with FastAPI expertise.' Instantly improves depth and accuracy of the response.

2. CONTEXT

Provide background information about your project, users, and constraints. More context leads to more tailored, accurate answers.

3. TASK

State clearly and specifically what you need done. Vague prompts give vague results — the more precise, the better the output.

4. CONSTRAINTS

Specify limitations — language version, library restrictions, length, style. Prevents answers that are technically correct but practically unusable.

5. FORMAT

Specify how the answer should be structured — code block, bullet points, JSON, explanation length. Controls exactly what you get back.

6. EXAMPLES (Optional)

Show 1-2 input-output examples to set the quality and format bar. Especially powerful for custom formatting and creative tasks.

Python Code — Advanced Prompting Patterns

```
import openai

client = openai.OpenAI(api_key='your-key')

# Few-Shot Prompting Example
few_shot_prompt = '''
Classify the sentiment. Respond with POSITIVE or NEGATIVE only.

Text: The food was absolutely amazing!
Sentiment: POSITIVE

Text: The service was terrible and slow.
Sentiment: NEGATIVE

Text: The movie exceeded all my expectations.
Sentiment:'''

# Chain of Thought
cot_prompt = '''
Q: If a train travels 120km in 2 hours, what is its speed?
A: Let me think step by step.
    Speed = Distance / Time
    Speed = 120km / 2 hours = 60 km/h
    The answer is 60 km/h.

Q: A car travels 240km in 3 hours. What is its speed?
A: Let me think step by step.'''

response = client.chat.completions.create(
    model='gpt-4o',
    messages=[{'role': 'user', 'content': cot_prompt}],
    temperature=0.0 # Deterministic for reasoning
)
print(response.choices[0].message.content)
```

What is RAG?

RAG (Retrieval Augmented Generation) is an architecture that enhances LLMs by giving them the ability to search and retrieve information from an external knowledge base before generating a response. It solves three critical LLM problems: stale training data, hallucinations, and inability to access private documents.

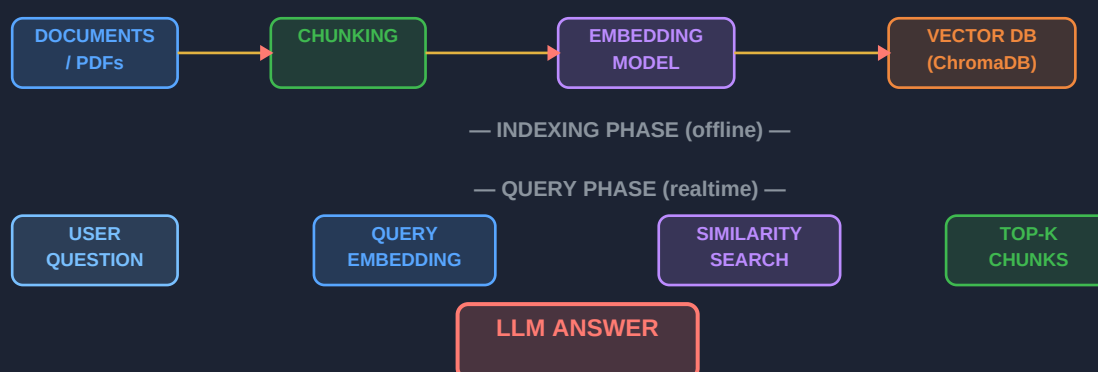
“

RAG is like giving a language model access to a search engine and a library at the same time — it can look things up before it speaks.

— Jerry Liu, LlamaIndex Co-founder

Complete RAG Architecture

Full RAG Pipeline — Indexing & Query Phases



RAG Components — All Definitions

Document Chunking

Splitting documents into smaller overlapping pieces (typically 500 tokens) before embedding. Chunk quality directly impacts retrieval accuracy.

Embedding Model

Converts text into numerical vectors that represent meaning — similar texts get similar vectors, enabling semantic search.

Vector Database

Stores embeddings and enables fast similarity search — finds relevant documents by meaning, not just keywords. Examples: ChromaDB, Pinecone, FAISS.

Cosine Similarity

Measures how similar two vectors are — from -1 (opposite) to 1 (identical). Used to rank retrieved document chunks by relevance.

Semantic Search

Finds documents by meaning similarity, not exact keyword match — powered by embeddings. Search 'heart attack' and find 'myocardial infarction' results.

Retriever

Converts the user's query to an embedding and returns the top-K most similar document chunks from the vector database.

Context Injection

Inserting retrieved document chunks into the LLM prompt — grounds the model's answer in your actual data, reducing hallucination.

Reranker

A second model that re-scores retrieved chunks for better precision — optional but significantly improves answer quality.

Advanced RAG Techniques

Naive RAG

The basic RAG pipeline: chunk → embed → retrieve → inject → generate. A solid starting point — build this first, then optimize.

HyDE (Hypothetical Document Embeddings)

Generate a hypothetical answer first, then use that as the search query — answers match documents better than questions do, improving retrieval.

Hybrid RAG

Combines semantic (embedding) search and keyword (BM25) search — catches both meaning-based and exact-match results for better coverage.

Graph RAG (Microsoft)

Builds a knowledge graph from documents — enables multi-hop reasoning across connected entities. Great for complex questions spanning many sources.

Agentic RAG

The AI decides when to retrieve, what to search, and whether results are sufficient — self-corrects by re-querying if the answer is not good enough.

COSINE SIMILARITY

$$\text{sim}(\mathbf{A}, \mathbf{B}) = (\mathbf{A} \cdot \mathbf{B}) / (||\mathbf{A}|| * ||\mathbf{B}||)$$

A,B=embedding vectors | Range: -1 (opposite) to 1 (identical)

Python Code — RAG System with LangChain + ChromaDB

```
# pip install langchain chromadb openai sentence-transformers
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI
from langchain.document_loaders import PyPDFLoader

# 1. Load documents
loader = PyPDFLoader('your_document.pdf')
documents = loader.load()

# 2. Chunk the documents
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50
)
chunks = splitter.split_documents(documents)

# 3. Embed and store in vector DB
embeddings = HuggingFaceEmbeddings(
    model_name='all-MiniLM-L6-v2'
)
vectordb = Chroma.from_documents(chunks, embeddings)

# 4. Create RAG chain
llm = ChatOpenAI(model='gpt-4o', temperature=0)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vectordb.as_retriever(search_kwargs={'k': 5})
)

# 5. Ask questions
answer = qa_chain.run('What is the main topic of this document?')
print(answer)
```


What is an AI Agent?

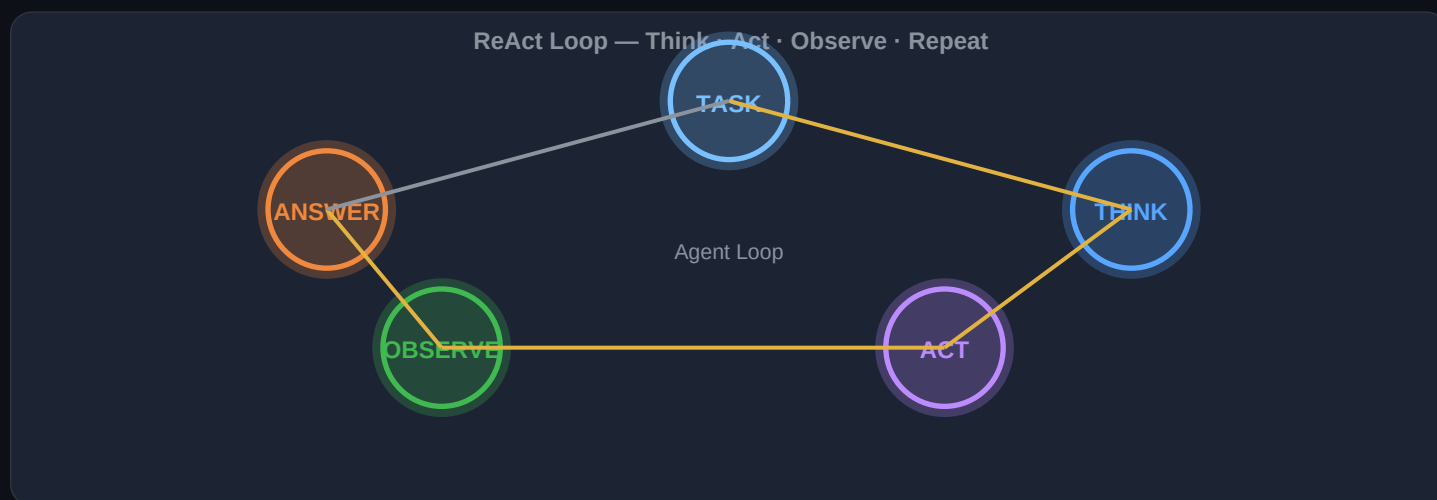
An AI Agent is an LLM that can autonomously plan, use tools, take actions, and complete multi-step tasks without human intervention at each step. Agents perceive their environment, reason about what to do, act using tools, observe results, and repeat — in a loop.

“

AI agents are AI systems that can take actions in the world on your behalf — browsing the web, writing code, managing files, and much more.

— Dario Amodei, Anthropic CEO

The ReAct Agent Loop



Planning

The agent decomposes a complex task into ordered sub-goals and determines the best sequence to complete them.

Tool Use / Function Calling

The LLM outputs structured calls to external functions — web search, code execution, databases — and uses the real results to answer.

Memory

Short-term: current conversation context. Long-term: external database storing facts across sessions. Gives agents continuity and recall.

Multi-Agent Systems

Multiple specialized agents collaborate — each handles what it is best at. Example: Planner → Researcher → Writer → Editor, each passing work along.

Model Context Protocol (MCP)

An open standard by Anthropic that lets any AI model connect to any tool or data source using a single consistent interface — like USB for AI.

Advanced LLM Concepts

Parameters (Weights)

The learnable numbers inside a neural network that encode all its knowledge. GPT-3 has 175 billion — more parameters means more capacity.

Pre-training

Training on trillions of tokens from the internet to learn language, facts, and reasoning. Extremely expensive — only done by large AI labs.

Instruction Fine-tuning (SFT)

Trains the pre-trained model on (instruction, ideal response) pairs — turns a text predictor into a helpful assistant.

Constitutional AI (Anthropic)

Anthropic's safety method where the AI critiques and revises its own outputs against a set of written principles — scalable and effective.

Mixture of Experts (MoE)

Only a subset of specialist sub-networks activates per input — allows huge model capacity while keeping inference fast and efficient. Used in GPT-4 and Mixtral.

Responsible AI — Ethics

Hallucination

Model generates confident false statements. Mitigation: RAG, source grounding, lower temperature, fact-checking layers.

Bias

AI inherits prejudices from its training data — can reflect stereotypes unfairly. Mitigated through diverse data, bias audits, and diverse human feedback.

Privacy

LLMs can memorize and inadvertently leak sensitive training data. Fine-tuning on private data requires careful data filtering and privacy-preserving techniques.

Jailbreaking

Prompting tricks that attempt to bypass an AI's safety guardrails. Mitigated through Constitutional AI, red-teaming, and adversarial training.

Energy Consumption

Training large AI models uses massive compute and electricity. The industry is moving toward efficient architectures, smaller models, and green data centers.

Python Code — AI Agent with LangChain

```
# pip install langchain openai duckduckgo-search
from langchain.agents import create_react_agent, AgentExecutor
from langchain.tools import DuckDuckGoSearchTool
from langchain.tools import tool
from langchain_openai import ChatOpenAI
from langchain import hub

# Define custom tool
@tool
def calculate(expression: str) -> str:
    '''Evaluate a math expression. Input: math expression string.'''
    return str(eval(expression))

# Tools list
tools = [DuckDuckGoSearchTool(), calculate]

# Create ReAct agent
llm = ChatOpenAI(model='gpt-4o', temperature=0)
prompt = hub.pull('hwchase17/react')
agent = create_react_agent(llm, tools, prompt)

# Run agent
agent_executor = AgentExecutor(
    agent=agent, tools=tools, verbose=True
)
result = agent_executor.invoke({
    'input': 'Search for the current NVIDIA stock price and calculate 100 shares value'
})
print(result['output'])
```

The GenAI Developer Stack

The modern GenAI developer stack has 5 layers, from foundational compute infrastructure up to user-facing applications. Understanding the full stack enables you to choose the right tool for each job and build production-grade AI systems.

“

Every company is becoming an AI company. The question is not whether to use AI — it's how fast you can build with it.

— Jensen Huang, NVIDIA CEO

Layer 1 — Foundation Libraries

Python

The go-to language for all AI and ML work — readable, versatile, and backed by the richest ecosystem of AI libraries.

NumPy

Foundational library for numerical computing — fast arrays, linear algebra, and matrix operations. The base that PyTorch and TensorFlow are built on.

PyTorch

Meta's deep learning framework — the most popular choice in AI research. Build and train any neural network with flexible, Pythonic code.

TensorFlow / Keras

Google's deep learning framework — great for production deployment. Keras provides a high-level API; TFLite enables models on mobile devices.

Layer 2 — Models & Inference

Hugging Face Transformers

The go-to hub for 500K+ pre-trained models — load any model in two lines of code. Also provides training, fine-tuning, and deployment tools.

Groq API

Ultra-fast LLM inference using custom hardware (LPU) — 10-20x faster than standard APIs. Perfect for real-time, latency-sensitive AI applications.

Ollama

Run open-source LLMs locally with one command — fully offline, free, and private. No API key or cloud required. Great for development and privacy.

Together AI

Cloud inference for open-source models at low cost — a cheaper alternative to OpenAI for LLaMA, Mistral, and image generation models.

Layer 3 — Storage

ChromaDB

The easiest local vector database — up and running in 3 lines of Python. Perfect for RAG prototypes and development. Free and open source.

Pinecone

Production-grade cloud vector database that scales to billions of vectors — the standard choice when moving AI apps from prototype to production.

FAISS (Facebook AI Similarity Search)

Meta's ultra-fast vector search library for local use — no server needed. Best when you need maximum speed on a single machine.

pgvector

Adds vector storage and similarity search to PostgreSQL — store AI embeddings alongside regular data in one database. No separate vector DB needed.

Layer 4 — Frameworks

LangChain

The most popular framework for building LLM-powered apps — chains together prompts, models, tools, memory, and retrievers with pre-built components.

LlamaIndex

A data framework for connecting LLMs to your own documents — simpler than LangChain when your main goal is RAG.

CrewAI

Framework for building multi-agent teams — define each agent's role and goal, then let them collaborate autonomously on complex tasks.

LangGraph

Builds agent workflows as graphs — gives precise control over flow, loops, and error handling. Industry standard for production-grade AI agents.

Layer 5 — Applications & DevOps

Streamlit

Turns any Python AI script into a web app in minutes — no HTML or CSS needed. Ideal for demos, dashboards, and rapid prototypes.

FastAPI

High-performance Python framework for building REST APIs — the standard for serving ML models in production. Auto-generates documentation.

Weights & Biases (W&B;)

MLOps platform for tracking training experiments — logs metrics, visualizes curves, and compares runs. Industry standard for organized ML development.

Docker

Packages your AI app with all its dependencies into a portable container — runs identically on any machine. Essential for reliable AI deployment.

Python Code — Full Stack: API + Streamlit UI

streamlit · AI Document Q&A App

```
# pip install streamlit openai langchain chromadb
import streamlit as st
import openai

# Streamlit app
st.title('AI Document Q&A')
st.sidebar.header('Settings')
temp = st.sidebar.slider('Temperature', 0.0, 1.0, 0.3)

# File upload
uploaded = st.file_uploader('Upload PDF', type=['pdf'])

if uploaded:
    st.success(f'Loaded: {uploaded.name}')
    question = st.text_input('Ask a question about the document:')

    if st.button('Get Answer') and question:
        with st.spinner('Thinking...'):
            # (RAG pipeline runs here)
            response = openai.OpenAI().chat.completions.create(
                model='gpt-4o',
                messages=[{'role': 'user', 'content': question}],
                temperature=temp
            )
            answer = response.choices[0].message.content
            st.markdown(f'### Answer\n{answer}')
```

All Important Formulas — Quick Reference

LINEAR REGRESSION

$$y = W \cdot x + b$$

W=weight matrix b=bias y=predicted output

SIGMOID (BINARY CLASSIFICATION)

$$\text{sigma}(z) = 1 / (1 + e^{(-z)})$$

Output: 0 to 1 | $z = w \cdot x + b$

SOFTMAX (MULTI-CLASS)

$$P(\text{class}_i) = e^{x_i} / \text{Sum}_j(e^{x_j})$$

Converts raw scores to probabilities summing to 1

ReLU ACTIVATION

$$f(x) = \max(0, x)$$

Most used hidden layer activation | Avoids vanishing gradient

MSE LOSS (REGRESSION)

$$L = (1/n) * \text{Sum}(y_i - \hat{y}_i)^2$$

y_i =actual $\hat{y}_i$ =predicted n =samples

CROSS-ENTROPY LOSS (CLASSIFICATION)

$$L = -\text{Sum}(y_i * \log(\hat{y}_i))$$

y_i =true one-hot $\hat{y}_i$ =predicted probability | Lower is better

GRADIENT DESCENT

$$w := w - \alpha * dL/dw$$

α =learning rate dL/dw =gradient | Minimize loss iteratively

TRANSFORMER ATTENTION

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) * V$$

Q=Query K=Key V=Value d_k =key dimension | Core of all LLMs

GAN MINIMAX

$$\min_G \max_D [E[\log D(x)] + E[\log(1-D(G(z)))]]$$

G=Generator D=Discriminator x =real z =noise

COSINE SIMILARITY (RAG)

$$\text{sim}(A, B) = (A \cdot B) / (||A|| * ||B||)$$

Range -1 to 1 | Used in vector DB retrieval

F1 SCORE

$$F1 = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

Best metric for imbalanced datasets | Harmonic mean

PRECISION & RECALL

$$\text{Precision} = TP / (TP + FP) \quad \text{Recall} = TP / (TP + FN)$$

TP=True Positive FP=False Positive FN=False Negative

Libraries Quick Reference

numpy

pip install numpy — Fast arrays and math operations. The foundation every other AI library is built on.

pandas

pip install pandas — Data loading, cleaning, and analysis. Essential for working with CSVs and tabular datasets.

scikit-learn

pip install scikit-learn — Classic ML algorithms, preprocessing, and evaluation metrics — all in one package.

torch

pip install torch — PyTorch for building and training neural networks. GPU-ready and used by most AI researchers.

transformers

pip install transformers — Access 500K+ pre-trained Hugging Face models — load in 2 lines of code.

openai

pip install openai — Official library for GPT-4o, DALL-E image generation, and Whisper speech-to-text.

anthropic

pip install anthropic — Official library for Claude models. Access Claude 3.5 Sonnet via simple API calls.

langchain

pip install langchain — Build RAG pipelines, agents, and chatbots with pre-built, composable components.

chromadb

pip install chromadb — Simplest local vector database — ready in 3 lines for RAG prototyping.

sentence-transformers

pip install sentence-transformers — Generate text embeddings locally using pretrained models. Free and fast.

diffusers

pip install diffusers — Hugging Face library for Stable Diffusion and other image/video generation models.

streamlit

pip install streamlit — Turn Python scripts into web apps instantly. No HTML/CSS needed — perfect for AI demos.

fastapi

pip install fastapi — Build production REST APIs to serve AI models. Fast, modern, and auto-generates documentation.

“

The best time to learn AI was five years ago. The second best time is right now.

— Community Saying

END OF GUIDE — 2025–2026 GENERATIVE AI EDITION